

Air Quality Prediction System

Machine Learning Regression Project with Streamlit

GitHub README Documentation

Developed by **Mohana Ranjani**

localhost:8501

Deploy



Air Quality Prediction System

This application uses a Machine Learning Decision Tree Regression model to predict CO(GT) concentration.

Enter Air Quality Measurements

PT08.S1(CO) 1000.00 - +	NOx(GT) 200.00 - +	PT08.S5(O3) 1000.00 - +
NMHC(GT) 100.00 - +	PT08.S3(NOx) 800.00 - +	Temperature (T) 25.00 - +
C6H6(GT) 10.00 - +	NO2(GT) 100.00 - +	Relative Humidity (RH) 50.00 - +
PT08.S2(NMHC) 900.00 - +	PT08.S4(NO2) 1000.00 - +	Absolute Humidity (AH) 1.00 - +

Streamlit application – Air Quality Prediction System

1. Project Overview

The **Air Quality Prediction System** is a machine learning application that predicts **CO(GT) (Carbon Monoxide)** concentration from air-quality sensor measurements and environmental parameters. A **Decision Tree Regressor** is trained on the Air Quality dataset and the trained model is integrated into a **Streamlit** web application.

2. Objective

The objective of this project is to build a regression model that learns patterns from air-quality measurements and provides a CO(GT) prediction for user-provided input values through an easy-to-use web interface.

3. Dataset

The project uses the **Air Quality UCI** dataset in Excel format. The notebook loads **AirQualityUCI.xlsx** and uses sensor measurements and environmental conditions as input features.

Input Feature	Role
PT08.S1(CO)	CO sensor response
NMHC(GT)	Non-methane hydrocarbons
C6H6(GT)	Benzene-related measurement
PT08.S2(NMHC)	NMHC sensor response
NOx(GT)	Nitrogen oxides
PT08.S3(NOx)	NOx sensor response
NO2(GT)	Nitrogen dioxide
PT08.S4(NO2)	NO2 sensor response
PT08.S5(O3)	Ozone sensor response
T	Temperature
RH	Relative humidity
AH	Absolute humidity
CO(GT)	Target – Carbon Monoxide concentration

4. Machine Learning Model

Model: Decision Tree Regressor

Prediction target: CO(GT)

Train/Test split: 80% training and 20% testing

random_state: 42

5. Project Workflow

1. Load the Air Quality dataset.
2. Select the 12 sensor/environmental input features.
3. Set CO(GT) as the target variable.
4. Split the data into training and testing sets.
5. Train a Decision Tree Regressor.
6. Evaluate the trained model using MAE, MSE and R².
7. Save the trained model with Joblib.
8. Load the saved model in Streamlit.
9. Accept air-quality measurements from the user and perform prediction.

6. Technologies and Libraries

Technology / Library	Purpose
Python	Programming language
Pandas	Data loading and manipulation
Scikit-learn	Machine learning and evaluation
DecisionTreeRegressor	CO(GT) regression model
Joblib	Saving and loading the trained model
Streamlit	Web application interface

7. Model Performance

Evaluation Metric	Value
Mean Absolute Error (MAE)	20.40328601953602
Mean Squared Error (MSE)	4013.1668136534104
R ² Score	0.32073718064277734

A sample prediction recorded in the notebook was **CO(GT) = 1.6**. The Streamlit screenshots also show that the prediction operation completed successfully.

8. Model Saving

After training, the Decision Tree model is saved using Joblib as **decision_tree_co_model.pkl**. This saved model is then used by the Streamlit application for prediction.

9. Streamlit Application

The Streamlit interface is titled **Air Quality Prediction System**. It provides input fields for the air-quality measurements and a **Predict CO(GT)** button. The application displays a success message after the prediction operation is completed.

Run the Streamlit application

```
streamlit run app.py
```

10. Streamlit Screenshots

The following screenshots show the actual Streamlit interface used for this project.

Screenshot 1

localhost:8501

Deploy



Air Quality Prediction System

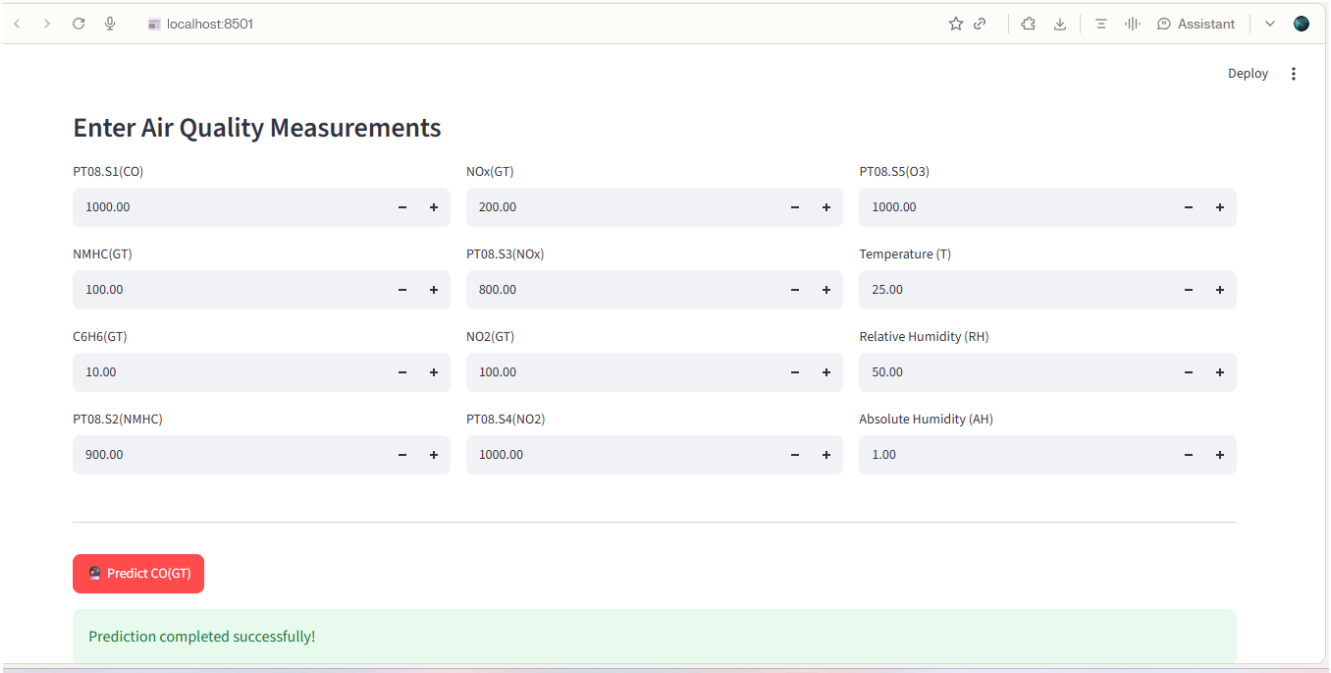
This application uses a Machine Learning Decision Tree Regression model to predict CO(GT) concentration.

Enter Air Quality Measurements

PT08.S1(CO) 1000.00 - +	NOx(GT) 200.00 - +	PT08.S5(O3) 1000.00 - +
NMHC(GT) 100.00 - +	PT08.S3(NOx) 800.00 - +	Temperature (T) 25.00 - +
C6H6(GT) 10.00 - +	NO2(GT) 100.00 - +	Relative Humidity (RH) 50.00 - +
PT08.S2(NMHC) 900.00 - +	PT08.S4(NO2) 1000.00 - +	Absolute Humidity (AH) 1.00 - +

10. Streamlit Screenshots (continued)

Screenshot 2



11. Project Structure

```
Air-Quality-Prediction/  
■■■ Air Quality Prediction.ipynb  
■■■ AirQualityUCI.xlsx  
■■■ decision_tree_co_model.pkl  
■■■ app.py  
■■■ README.md  
■■■ screenshots/
```

12. How to Run the Project

1. Install Python and the required libraries.
2. Keep the dataset and trained model in the project folder.
3. Run the notebook to train and save the model, if the model file is not already available.
4. Open the project folder in Command Prompt or VS Code terminal.
5. Run the Streamlit application using the command below.
6. Enter the air-quality measurements in the Streamlit input fields.
7. Click **Predict CO(GT)** to generate the prediction.

13. Requirements

Install the main dependencies with:

```
pip install pandas scikit-learn joblib streamlit openpyxl
```

14. Future Enhancements

- Improve prediction performance using hyperparameter tuning.
- Compare Decision Tree with Random Forest and other regression algorithms.
- Add charts for air-quality trends and model performance.
- Add input validation and clearer prediction explanations.
- Deploy the Streamlit application online.

15. Conclusion

This project demonstrates a complete machine learning workflow for predicting CO(GT): dataset preparation, feature selection, model training, evaluation, model serialization, and Streamlit-based deployment. The application provides a simple interface for entering air-quality measurements and performing a CO(GT) prediction.